

DATABASE DEVELOPMENT

Configure MongoDB

MongoDB Atlas was selected as the cloud database for the Stock Trading application because it provides a secure, scalable, and highly available NoSQL database service. A free MongoDB Atlas account was created, followed by a new project and cluster. A database user was created with authentication credentials, and the local IP address was added to the network access list. The Node.js connection string was generated, the password placeholder was replaced with the actual database password using URL encoding where required, and the final connection string was stored securely in the .env file as MONGO_URI. This configuration protects sensitive information while enabling secure communication between the backend application and the cloud database.

Create Database Connection

A database configuration file named db.js was created in the config folder. Mongoose was used to connect the Express server to MongoDB Atlas. The connectDB() function uses mongoose.connect(process.env.MONGO_URI) to establish the connection. When the connection succeeds, the application displays the connected database host in the terminal. If the connection fails, an appropriate error message is shown and the application exits to avoid processing requests without a database. The connectDB() function is imported into server.js and executed before the Express server starts, ensuring that the backend always has an active database connection.

Create Schema and Models

Schemas and models were created using Mongoose to define the database structure. A User schema was implemented with fields such as name, email, and password. Validation rules including required fields and unique email addresses were added to improve data integrity. The schema was converted into a User model using mongoose.model(), allowing controllers to perform Create, Read, Update, and Delete (CRUD) operations. The User model is used by the registration and login controllers to store and retrieve user information efficiently. This model-based approach keeps the backend modular, maintainable, and aligned with the MVC architecture.